

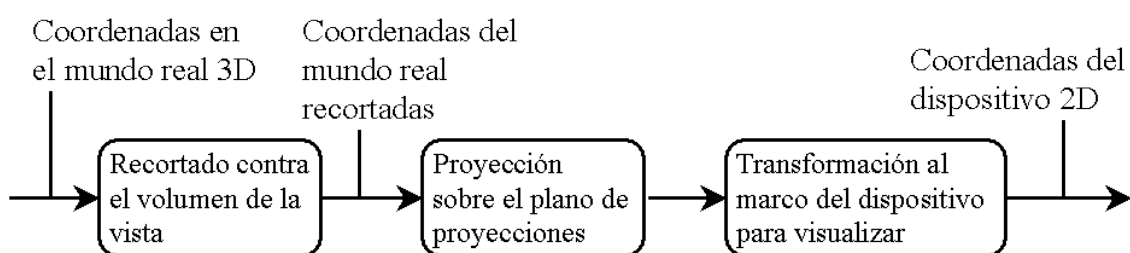
Terminales raster, Memoria de refresco y el color en los terminales raster

A principio de los 70, la difusión de la televisión lleva a aplicar su tecnología para el desarrollo de las llamadas terminales de barrido o **terminales raster**. En estas terminales también existe un direccionamiento de puntos como una matriz bidimensional pero ahora el haz no la recorre libremente sino de forma ordenada desde la parte superior izquierda hasta la inferior derecha (monitores no entrelazados) dando a cada punto, que a partir de ahora denominaremos píxel, el color, la intensidad y el brillo que hay almacenado en la **memoria de refresco**. En esta arquitectura la memoria de refresco no almacena la forma en la que se recorre la pantalla para hacer la imagen sino los valores de cada punto de la pantalla de forma que el modo en el que esta se recorre es siempre fijo y queda establecido por el hardware del dispositivo.

La memoria de refresco es la imagen bitmap más sencilla y sería un array de unos y ceros indicando que píxeles de la pantalla están encendidos o apagados, de forma que ésta visualizará una imagen monocromo sin niveles de gris. Para disponer de una imagen en color, necesitaremos uno de estos arrays por cada plano de color formándose de este modo imágenes con 8 valores correspondientes a la mezcla de los 3 colores básicos con su máxima intensidad.

Si extrapolamos este planteamiento con el color implicaría 8 bits para cada color lo que daría como resultado 2^{24} combinaciones de colores posibles, lo que habitualmente se denomina color real. Para determinar la memoria que necesitamos para almacenar cada una de estas imágenes deberemos calcular el número de píxeles de la imagen y luego multiplicarlos por el número de bits que necesitamos para representar cada píxel (1 bit para el monocromo, 3 para 16 colores, 8 para 256 niveles de gris y 24 para color real), para obtener el resultado en bytes sólo tendremos que dividir entre 8.

Haz u esquema de la visión 3D, explícalo



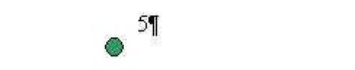
En este sentido el proceso de visualización llevado a cabo por la pipeline grafica se realiza mediante los siguientes pasos. El primer paso es la transformación de las coordenadas locales de objeto a las del “world” en el que se desarrolla la escena. Estas coordenadas son proyectadas obteniendo las coordenadas de proyección. Estas coordenadas son ya transformadas mediante técnicas de proyección 2D en las del device.

Que es un modelo poliédrico, que debe de tener en cuenta.

Consiste en definir el objeto mediante una aproximación a su superficie mediante polígonos, en general triángulos, en un proceso que se denomina *tesselation*. Dichos polígonos comparten sus aristas y vértices, es decir, que un poliedro, que puede ser abierto o cerrado. La exigencia de que sea una variedad bidimensional se traduce, en este caso, en la restricción de que los polígonos no se intersequen entre sí excepto en las aristas, en una arista no puedan confluir más de dos polígonos y un vértice debe pertenecer al menos a una arista. En un vértice pueden coincidir cualquier número de polígonos. Además, una buena representación poligonal debe tener en cuenta:

- Evitar la degeneración de los polígonos, que se produce por ejemplo, si uno de sus lados tiene longitud cero. Los triángulos degenerados son especialmente problemáticos, ya que resulta un objeto de área nula, normalmente invisible, pero que puede provocar errores en ciertos algoritmos.
- Evitar los vértices en T ya que puede provocar problemas a la hora de rellenar el color por interpolación entre los vértices y otras singularidades en ciertos algoritmos.
- Utilizar en lo posible primitivas poligonales que permitan especificar el objeto sin repetir varias veces los vértices que pertenezcan a varios polígonos.
- Falta de coplanariedad en polígonos de mas de 4 lados.

1) Que obtienes si a partir de los siguientes puntos si ejecutas los siguiente comandos con `glPolygonMode(GL_FRONT, GL_FILL)`, dibújalo.

	<pre>glBegin(GL_TRIANGLES);</pre> 1) secuencia, 1,2,3,4,5 2) secuencia 3,2 1, 3,4,5 3) secuencia 2,1,3,4,3,2,5,4,3. <pre>glBegin(GL_TRIANGLE_STRIP).</pre> 1) secuencia, 1,2,3,4,5 2) secuencia 2,1,3, 4,5
---	---

